# Atelier 7 : GitLab CI/CD + SonarQube Cloud

Compte rendu des étapes réalisées

## Étape 1 : Créer le projet et le code V1

- Création du projet GitLab `atelier7` et clonage en local.
- Création de `index.html`, `style.css` et `script.js` (V1 correcte, avec `textContent`).

```
bouton.addEventListener("click", function () {
  message.textContent = "Le JavaScript fonctionne correctement !";
});
```

## Étape 2 : Préparer SonarQube Cloud

- Connexion à SonarQube Cloud avec GitLab, puis import du projet (*Analyze new project*).
- Récupération de la Project Key et de l'Organization Key (*Project information*).
- Création du fichier `sonar-project.properties` :

```
sonar.projectKey=<organisation>_atelier7
sonar.organization=<organisation>
sonar.sources=.
sonar.exclusions=public/**
sonar.qualitygate.wait=true
```

## Étape 3 : Créer le token et les variables GitLab

- SonarQube Cloud : *My Account* → *Access tokens*, génération du token `gitlab-ci`.
- GitLab : *Settings* → *CI/CD* → *Variables*, ajout de `SONAR_TOKEN` et `SONAR_HOST_URL` (`https://sonarcloud.io`).

## Étape 4 : Créer le pipeline V1 et pousser le code

- Fichier `.gitlab-ci.yml` avec 2 jobs : `sonarqube-check` (stage quality) et `deploy-pages` (stage deploy).

```
git add .
git commit -m "V1 - correct code"
git push
```

## Étape 5 : Erreur « Automatic Analysis »

- Le premier pipeline (`#3053755405`) échoue : *You are running CI analysis while Automatic Analysis is enabled*.
- Solution : SonarQube Cloud → *Administration* → *Analysis method* → désactiver **Automatic Analysis**.
- GitLab → *Build* → *Pipelines* → *Retry* : le pipeline passe (`#3053770651`) et le Quality Gate est **Passed**.

## Étape 6 : Vérifier le déploiement GitLab Pages (V1)

- *Deploy* → *Pages* : le site est publié et actif.

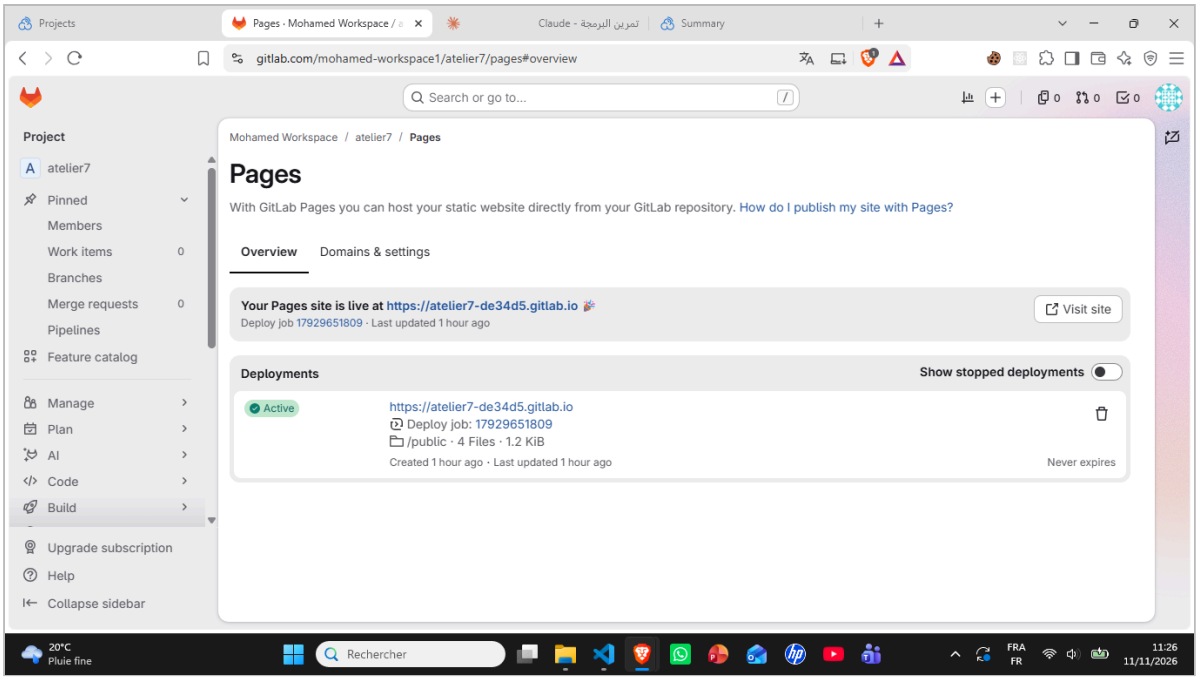



Figure 1 : GitLab Pages, site déployé

- Ouverture du site : affichage de la V1.

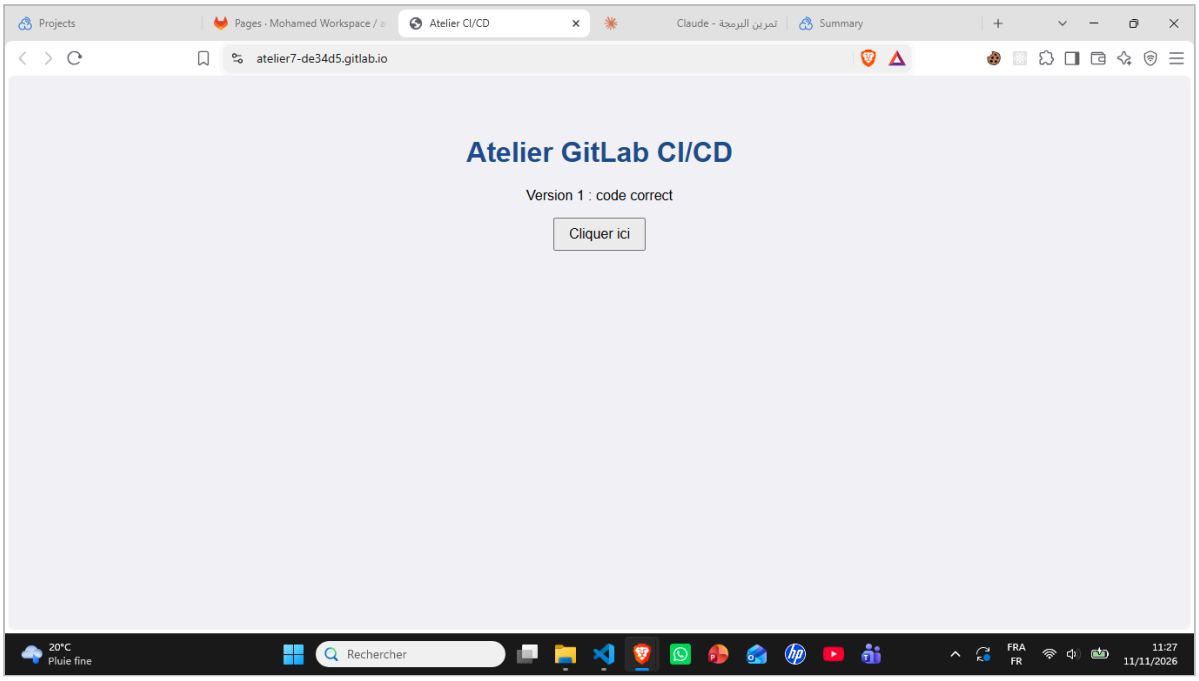



Figure 2 : Site V1 avant le clic

- Clic sur le bouton : le texte change, le JavaScript fonctionne.

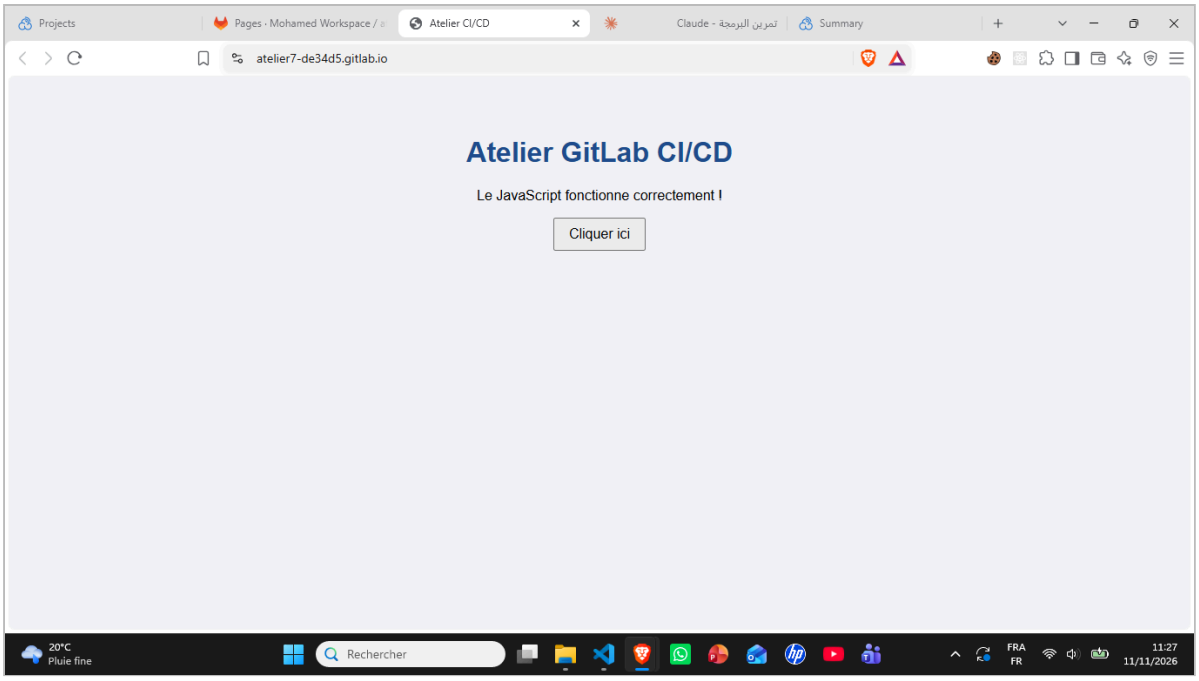



Figure 3 : Site V1 après le clic

## Étape 7 : Créer la V2 (injection XSS) et pousser

- `index.html` : ajout d'un formulaire (champ de saisie + bouton « Afficher »).
- `script.js` : la saisie est insérée avec `innerHTML` (vulnérable).
- `.gitlab-ci.yml` : 3 stages : **quality** ( `syntax-check` , `sonarqube-check` ), **security** ( `security-check` ), **deploy** ( `deploy-pages` ).

```
message.innerHTML = input.value;
```

```
git add .
git commit -m "V2 - injection xss"
git push
```

## Étape 8 : Résultat du pipeline V2

- Pipeline `#3054111288` : **Failed**.
- `syntax-check` réussi, `sonarqube-check` échoué ; les stages `security` et `deploy` ne sont pas exécutés (déploiement bloqué).

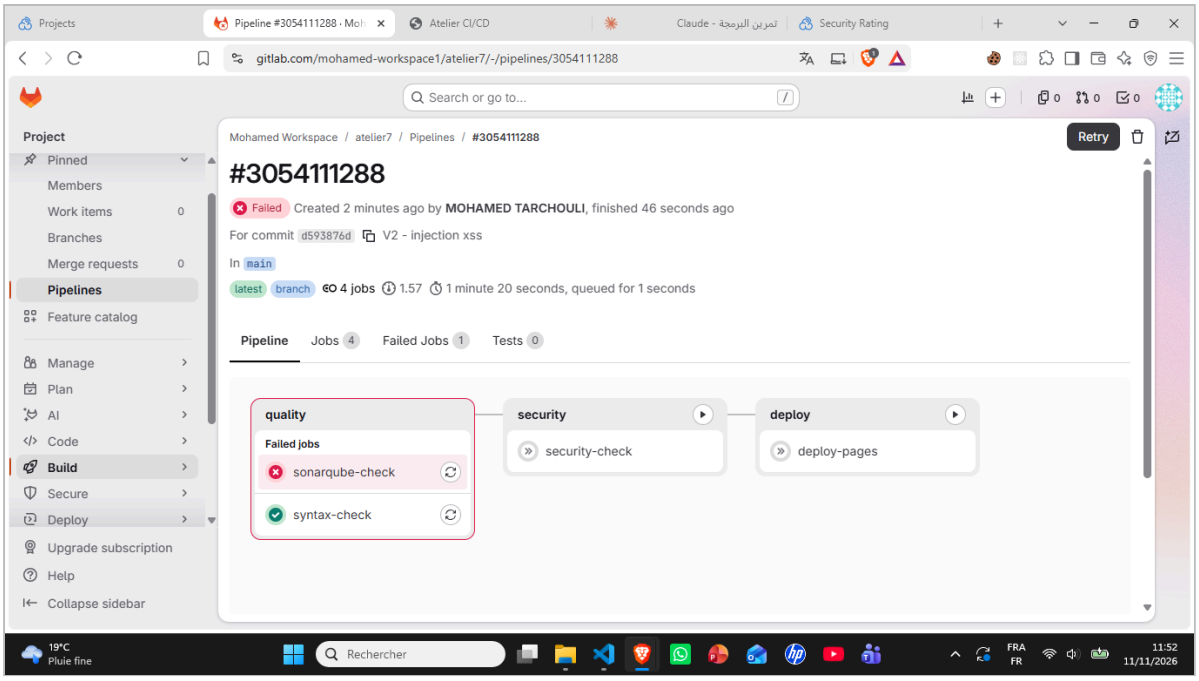



Figure 4 : Pipeline #3054111288 échoué au stage quality

## Étape 9 : Résultat dans SonarQube Cloud

- `script.js` : Security Rating **E**, 1 issue, effort de correction 30 min.
- Ligne concernée : `message.innerHTML = input.value;` (DOM XSS).

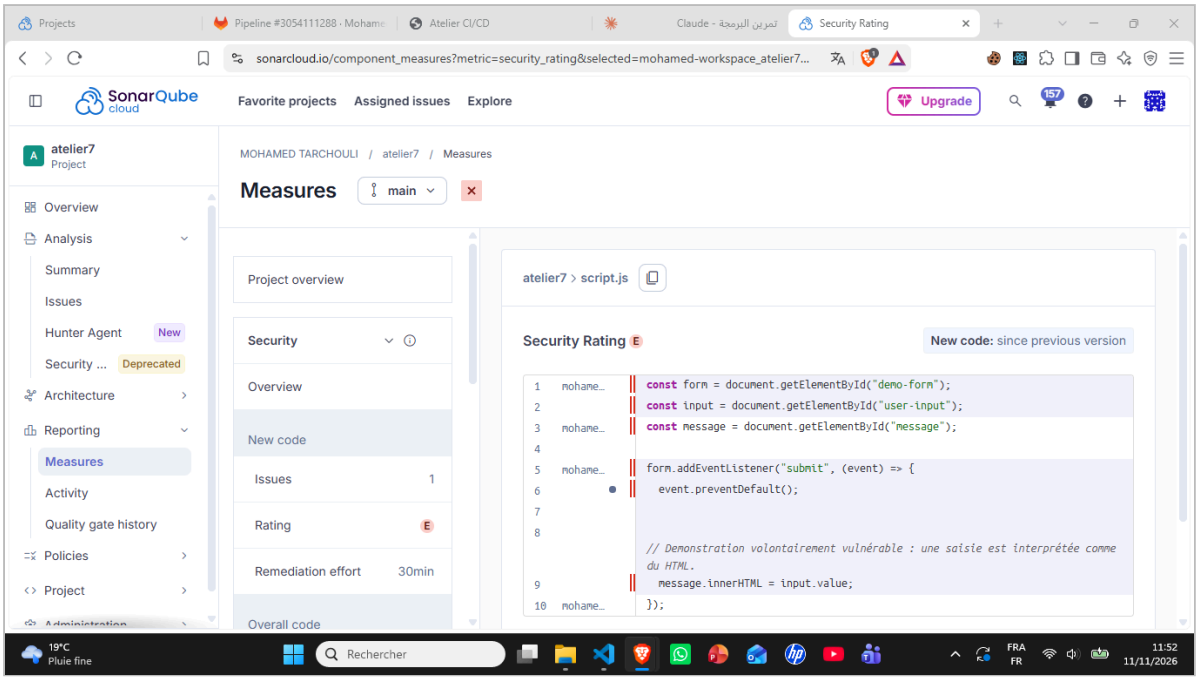



Figure 5 : SonarQube Cloud, vulnérabilité détectée

## Étape 10 : Historique des pipelines

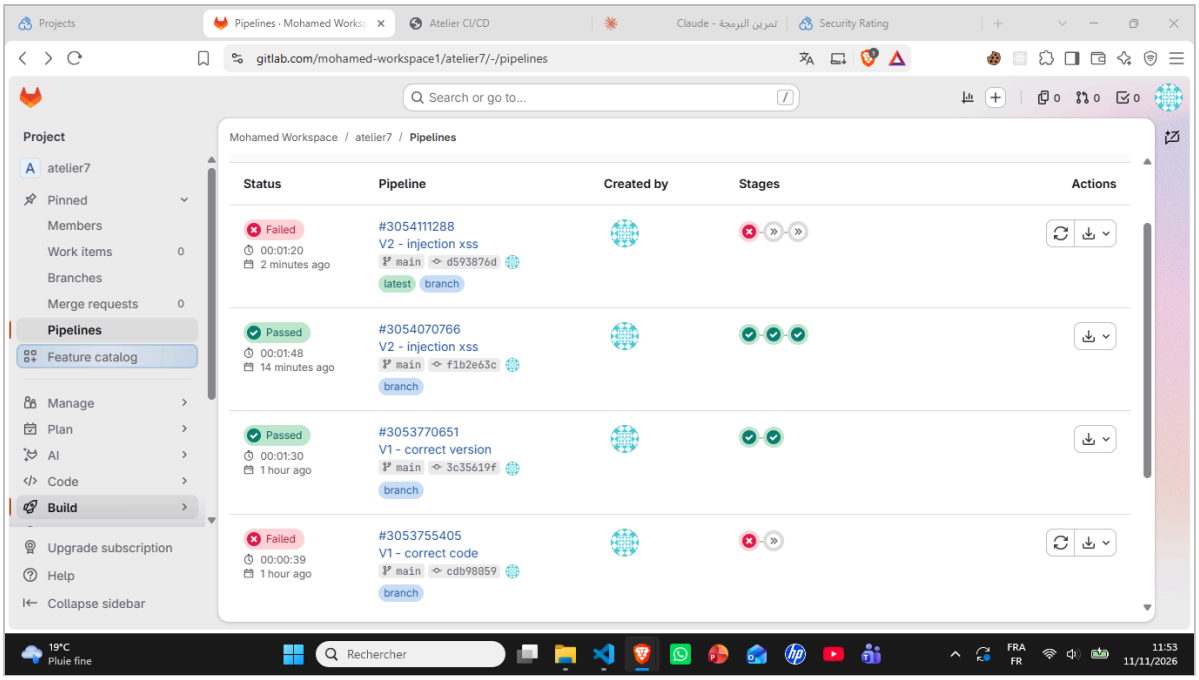



Figure 6 : Liste des pipelines sur la branche main